

I. ARDUINO

1. CONTENU DE LA VALISE :

a. Carte Arduino DFRduino UNO R3 + câble USB

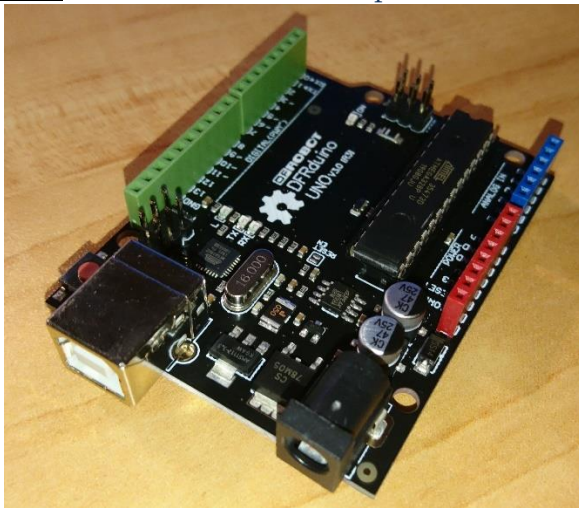
Un microcontrôleur est un système qui ressemble à un ordinateur : il a une mémoire, un processeur, des interfaces avec le monde extérieur. Les microcontrôleurs ont des performances réduites, mais sont de faible taille et consomment peu d'énergie, les rendant indispensables dans toute solution d'électronique embarquée (voiture, porte de garage, robots, ...).

La carte Arduino n'est pas le microcontrôleur le plus puissant, mais son architecture a été publiée en open-source, et toute sa philosophie s'appuie sur le monde du libre, au sens large.



La carte Arduino se relie à un ordinateur par un câble USB. Ce câble permet à la fois l'alimentation de la carte et la communication série avec elle.

Exo 1. Sur la carte Arduino, repérer les éléments suivants :



- Connecteur USB : com série alim5V / 500 mA
- Alimentation extérieure 7-12 V (optionnelle)
- Témoin d'alimentation
- Microcontrôleur
- Entrées analogiques
- Entrées/Sorties digitales

b. Base Shield V2 : connecteurs Analogiques A0..A3, digitaux D2..D8



Connecter le Base Shield à la carte en prenant garde de ne pas tordre les broches.

c. Leds : Rouge et Vert



Brocher les leds sur leur support carré, puis les câbles sur le support

d. Capteur de rotation d'angle

e. Capteur de lumière

f. Bouton



Vérifier que la valise est complète

2. SORTIES NUMERIQUES

a. cahier des charges :

Cdc 1. Faire clignoter la LED rouge connectée sur le PORT numérique n°2 avec une période de 1 seconde.

b. Matériel



Brancher le module LED rouge sur le connecteur 2.

Ce connecteur est une information numérique binaire (ou logique), qui ne prendra donc que les deux états 0 (LOW) ou 1 (HIGH). Lorsque l'état de ce connecteur vaudra 0, la led sera éteinte, et lorsqu'il vaudra 1, la led sera allumée.

c. Logiciel



Codage



Lancer l'application *Arduino* sur le PC, soit en cliquant sur l'icône sur le bureau, soit en lançant le programme à partir du menu *Démarrer*. Le programme s'ouvre sur l'éditeur de code.

Un programme ARDUINO comporte toujours la structure minimale suivante :

```
void setup()
{
  //fonction d'initialisation
}

void loop()
{
  //fonction principale qui tourne en boucle
}
```



La fonction **setup()** permet de configurer le matériel, elle n'est exécutée qu'une seule fois au lancement du programme.



La fonction **loop()** est le **programme principal**. Elle est exécutée après la fonction **setup()**, elle est exécutée automatiquement en boucle.

fonction **setup()**



La led est une sortie du logiciel : C'est le programme qui allume ou éteint cette led.

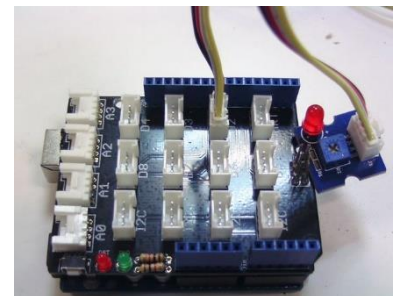
le microcontrôleur doit donc considérer le connecteur 2 sur lequel est connecté la led comme une sortie :

On utilise la primitive arduino suivante : **pinMode(n°de port, INPUT | OUTPUT)**

Ici n° de port prendra la valeur 2, suivi de OUTPUT pour la mettre en sortie

```
void setup()
{
  pinMode(2, OUTPUT) ;
}

void loop()
{
  //fonction principale qui tourne en boucle
}
```



la fonction `loop()`

- ✚ Pour faire clignoter la led, il suffit de faire en sorte que le programme vienne alterner les états 0 et 1 sur le connecteur 2 en temporisant entre chaque changement d'état.
On implémente donc dans la fonction `loop()` l'algorithme suivant :

```
Positionner la sortie 2 à '1';  
Temporisation (500ms);  
Positionner la sortie 2 à '0';  
Temporisation (500ms);
```

- ✚ Pour positionner la sortie 2 à '1' ou '0' (ou encore à 'VRAI' ou 'FAUX', ou à l'état 'HAUT' ou 'BAS'), on utilise la primitive arduino suivante :
`digitalWrite(n°de port, HIGH | LOW)`.
- ✚ La primitive **`delay(temps en ms)`** permet d'obtenir la temporisation. (le microcontrôleur ne fait rien, il attend le délai écoulé précisé en argument)
- ✚ Le code devient alors :

```
void setup()  
{  
  pinMode(2, OUTPUT) ;  
}  
  
void loop()  
{  
  digitalWrite(2, HIGH) ;  
  delay(500) ;  
  digitalWrite(2, LOW) ;  
  delay(500) ;  
}
```

 Exécution et Test

- ✚ Dans le menu Outils / Port série /
Sélectionner le dernier port de communication COMxx qui correspond au port USB sur lequel est branché la carte.



Le numéro de port change d'un poste de TP à l'autre !

- ✚ Enregistrer votre programme sous le nom « Led_CdC1 ».
- ✚ Cliquer sur  pour COMPILER / VERIFIER le code.
Corriger les erreurs de syntaxe éventuelles, ...
- ✚ Cliquer sur  pour télécharger le programme vers le microcontrôleur.
Attendre la fin du processus, et observer le résultat sur la carte... Debugger si besoin...



Si rien ne se passe, inverser la position des deux broches de la led sur sa carte

 Quelques améliorations pour la lisibilité/maintenabilité du code...

- ✚ Il est plus lisible utiliser un nom comme « `BROCHE_LED_ROUGE` » dans le code que d'utiliser le numéro 2.
On peut donc associer un « alias plus parlant » à la broche 2 de plusieurs manières, il suffit de déclarer tout en haut du sketch :
`#define BROCHE_LED_ROUGE 2`, ou bien :
`const int BROCHE_LED_ROUGE = 2`, ou bien encore :
`int BROCHE_LED_ROUGE = 2;`

Le code devient alors :

```
const int BROCHE_LED_ROUGE = 2;

void setup() {
  pinMode(BROCHE_LED_ROUGE, OUTPUT) ;
}

void loop() {
  digitalWrite(BROCHE_LED_ROUGE, HIGH) ;
  delay(500) ;
  digitalWrite(BROCHE_LED_ROUGE, LOW) ;
  delay(500) ;
}
```

d. cahier des charges option :



Modifier le code afin que le programme réponde au cahier des charges suivant.
Enregistrer le programme sous les noms « Led_CdC2 ».

CdC 2. Faire clignoter la LED rouge connectée sur le PORT numérique n°2 avec une période oscillant entre 10 ms et 500 ms avec un pas de 30 ms : Le clignotement est lent à 500 ms, il accélère jusqu'à atteindre une période de 10 ms puis ralentit à nouveau jusqu'à 500 ms, etc.

3. ENTREES NUMERIQUES

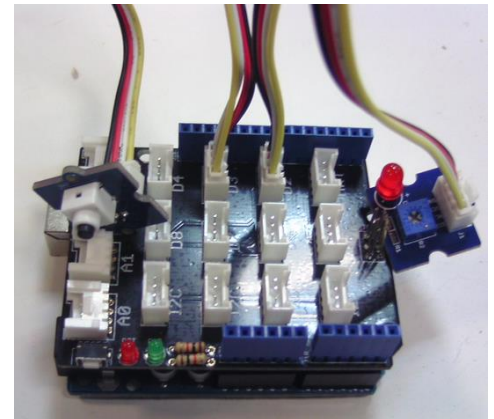
a. cahier des charges :

CdC 3. Ne faire clignoter la LED connectée sur le PORT numérique n°2 avec une fréquence de 1s que lorsque le bouton poussoir est enfoncé.

b. Configuration du Materiel



Brancher le module LED rouge sur le connecteur 2, et le module bouton poussoir sur le **connecteur 3**.
Lorsque le bouton poussoir est enfoncé, l'état logique du connecteur 3 est à '0' (LOW) ;
lorsqu'il ne l'est pas, l'état du connecteur est à '0' (HIGH).



c. Codage du Logiciel



Enregistrer votre premier programme «Led » sous le nouveau nom « LedButton_CdC3»

fonction **setup()**



Le bouton poussoir est **une entrée du logiciel** puisque le programme réagit **en fonction de l'état du bouton**.

Le microcontrôleur doit donc considérer le connecteur 3 sur lequel est connecté le bouton comme une **entrée**:



Compléter le `setup()` : on ré-utilise la primitive arduino:

`pinMode(n°de port, INPUT | OUTPUT)`

Ici n° de port prendra la valeur 3, suivi de INPUT pour la mettre en entrée.



On définit (et on y fait référence dans le code...) la constante `BROCHE_BOUTON` .

```
const int BROCHE_BOUTON = 3;
```

fonction `loop()`

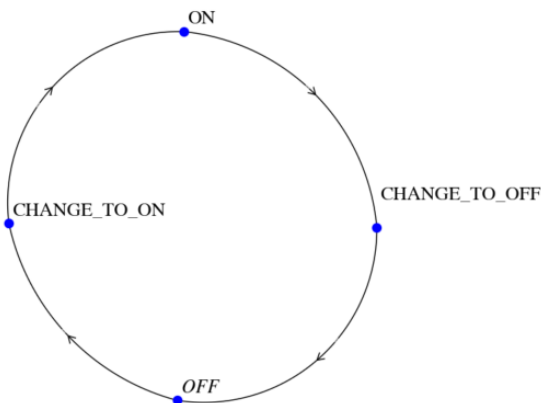
- ✚ Pour lire l'état du connecteur 3, on utilise la primitive arduino suivante : `int digitalRead(n° de port)`,
qui renvoie l'état de l'entrée HIGH ou LOW.

d. cahiers des charges option :

- ✚ Modifier le code afin que le programme réponde aux cahiers des charges suivants.
Enregistrer les programmes sous les noms « LedButton_CdC4 » et « LedButton_CdC5 »

CdC 4. Un premier appui sur le bouton déclenche le clignotement de la LED rouge connectée sur le PORT numérique n°2. L'appui suivant déclenche l'arrêt du clignotement.

- ✚ A implémenter avec un **graphe d'état** sur la variable `etatCligno` :
`etatCligno ∈ {ON;CHANGE_TO_OFF;OFF;CHANGE_TO_ON}`
Les transitions sont conditionnées à l'état du bouton.



CdC 5. Chaque appui sur le bouton provoque un changement de fréquence de clignotement. La période oscille entre 10 ms et 1000 ms avec un pas de 100 ms.

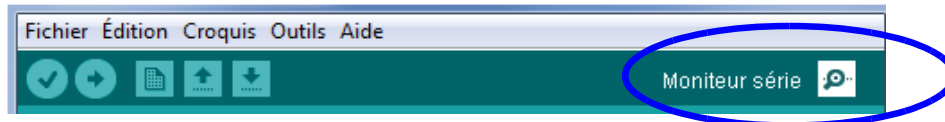
4. UTILISATION DE LA LIAISON SERIE

La carte ARDUINO possède une liaison série asynchrone qui permet de relier la carte au PC. Cette fonction permet outre la programmation de la carte elle-même, de faire dialoguer l'application programmée avec le PC de manière bidirectionnelle.

Cette partie décrit les outils disponibles pour faire communiquer ARDUINO avec le PC.

- ✚ La carte ARDUINO et le PC doivent être configurés au même débit de transmission :
- ✚ Coté ARDUINO,
 - Pour configurer le débit de transmission, il suffit d'appeler la fonction `Serial.begin(vitesse_de_transmission)` dans le `setup()` :
Les vitesses de transmission sont normalisées et les valeurs possibles en nombre de bits/s sont les suivantes :
300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, ou 115200.
La vitesse adaptée au quartz présent sur la carte est 9600 bits par seconde.
 - Pour envoyer une information au PC, on utilise la fonction `println()` aussi bien dans la fonction `setup()` que dans la fonction `loop()` :

- Sur le PC pour visualiser les informations en provenance de la carte ARDUINO, il suffit de lancer la console série intégrée à ARDUINO :



La fenêtre de la console apparaît alors, en affichant ce que la carte Arduino envoie sur la liaison



a. Transmission dans le sens Arduino→PC

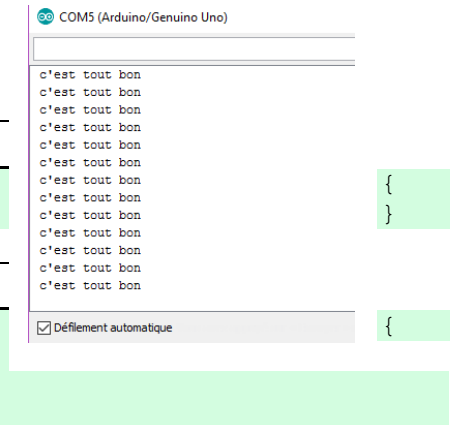
➡ Codage d'un test basique

fonction **setup()**

```
void setup()
  Serial.begin(9600) ;
```

fonction **loop()**

```
void loop()
  Serial.println("c'est tout bon") ;
  delay(500) ;
}
```



- Enregistrer le programme sous le nouveau nom « LSerie_Emission »

➡ Exécution et test

- La console fait apparaître les messages reçus toutes les 500 ms :

b. Transmission dans le sens PC→ Arduino

A partir de la console affichée sur le PC, il est possible de saisir un caractère dans la fenêtre d'édition, puis de l'envoyer vers la carte Arduino.

De son côté la carte Arduino doit tester la présence d'un ou de plusieurs caractères reçus en provenance du PC.

On utilisera la primitive `int Serial.available()` qui renvoie le nombre d'octets reçus.

Les caractères présents sont alors stockés sous forme d'octets par le driver dans une mémoire tampon (on parle d'un buffer de réception) accessibles à la carte Arduino. Le programme doit alors interpréter les octets présents sous forme de caractères, d'entiers, de décimal ou de chaînes de caractères en utilisant l'une des primitives suivantes :

- `char read()` qui renvoie un caractère,
- `int parseInt()` qui renvoie un entier,
- `float parseFloat()` qui renvoie un float,
- `String readString()` qui renvoie une chaîne.

➡ Codage d'un test basique

- Modifier le code de « LSerie_Emission » afin que le programme puisse recevoir et interpréter un octet sous forme d'entier :

Enregistrer ce programme sous le nom « LSerie_ReceptionEmission »

fonction **setup()**

```
void setup()
{
```

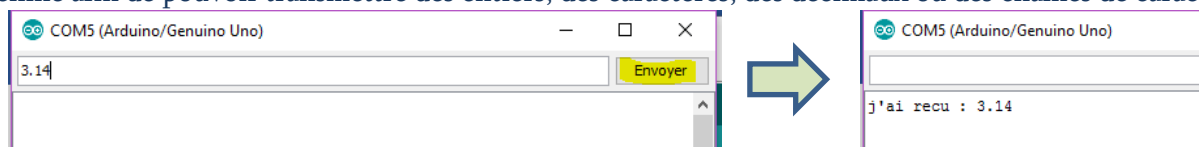
```
Serial.begin(9600) ;  
}
```

fonction **loop()**

```
void loop()  
{  
  if (Serial.available() > 0)  
  {  
    int num = Serial.parseInt();  
    Serial.print("j'ai reçu : ");  
    Serial.println(num);  
  }  
}
```

➡ Exécution et test

- Utiliser la fenêtre d'édition de la console pour saisir et envoyer un caractère.
- Modifier et tester les différentes fonctions de lecture et observer à chaque fois l'interprétation par la machine afin de pouvoir transmettre des entiers, des décimaux ou des chaînes de caractères.



5. UTILISATION DES ENTREES ANALOGIQUES

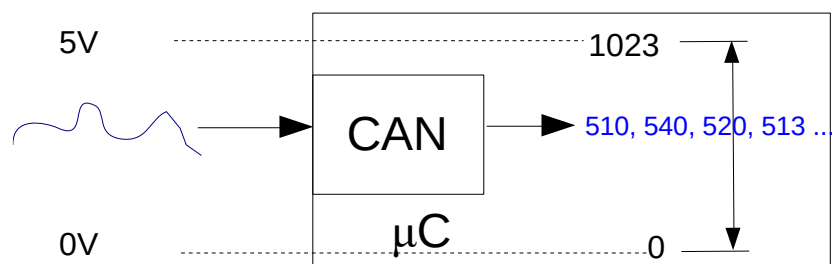
a. Convertisseur Analogique/Numérique

Le microcontrôleur ne connaît que des valeurs numériques, cependant certaines entrées sont équipées de convertisseurs analogique/numérique (CAN ou ADC en anglais pour Analog to Digital Converter), qui permettent notamment de connecter puis exploiter des capteurs qui fournissent une tension analogique.

La plage de conversion de la tension d'entrée analogique est comprise entre 0V et 5V. Le convertisseur interne fonctionnant sur 10 bits, la plage des valeurs numériques est donc comprise entre 0 et 1023 (donc 2^{10} valeurs possibles)

Ainsi 0V en entrée correspond à une valeur interne de 0, et 5V à 1023, on peut considérer que la loi de variation entre ces 2 valeurs est une fonction affine.

On pourra connecter n'importe quel capteur sur une des entrées analogiques à condition qu'il fournisse une tension comprise dans la plage précitée (0V - 5V).



Pour lire la valeur d'une entrée, on utilisera la fonction de lecture de l'entrée analogique :

int analogRead(n° d'entrée) qui renvoie une valeur de type int,
on stockera donc cette valeur dans une variable de même type.

b. Cahier des Charges

CdC 6. On mesure l'intensité lumineuse ambiante, à l'aide d'une photorésistance.

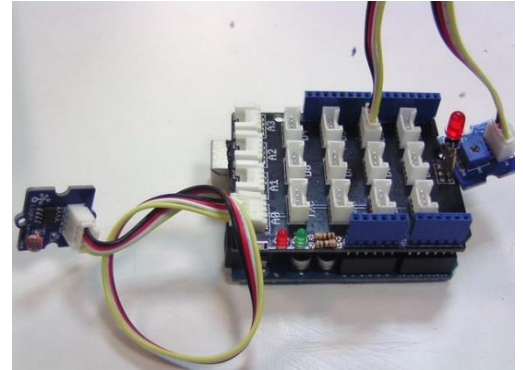
A la tombée de la nuit, lorsque l'intensité lumineuse est trop faible, on déclenche l'allumage des lumières (simulé par l'allumage de la led), lorsque le jour réapparaît (à partir d'un seuil) la lumière s'éteint, et ainsi de suite...



Pour info :

La photorésistance (LDR pour **L**ight **D**ependent **R**esistor) voit sa résistance varier de manière exponentielle, inversement à la lumière. On mesure une résistance supérieure à 10 MΩ, dans l'obscurité, et quelques kΩ en plein jour.

On insère la LDR dans un pont résistif diviseur de tension pour obtenir une tension qui varie en fonction de la lumière.
On obtient alors un capteur de lumière (Light Sensor)



Cette structure est déjà intégrée dans le module LDR .

c. Configuration du Materiel

Brancher le module LED rouge sur le connecteur 2, et le module LDR sur l'entrée analogique 0.

d. Recherche du seuil jour/nuit

A chaque intensité de lumière correspond une valeur fournie par le LDR. On recherche expérimentalement la valeur fournie par le LDR qui marque le basculement entre le jour et la nuit.
Cette recherche nécessite l'écriture d'un programme qui se contente de lire l'intensité lumineuse sur le LDR et qui en donne lecture à l'utilisateur en envoyant en envoyant cette valeur sur la liaison série.

La lecture sur l'entrée analogique du LDR se fait en utilisant la primitive arduino suivante :
`int analogRead(n°de port)`, qui renvoie la valeur de l'entrée analogique câblée sur le port n°de port.

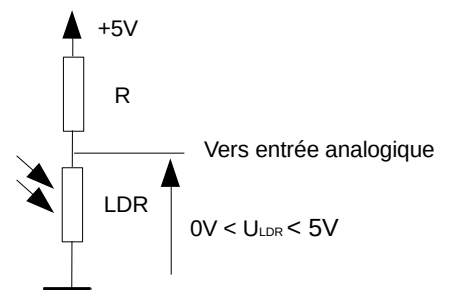
Exo 2.

a. La lecture peut donc se faire de la manière suivante :

```
void setup()
{
  Serial.begin(9600) ;
}

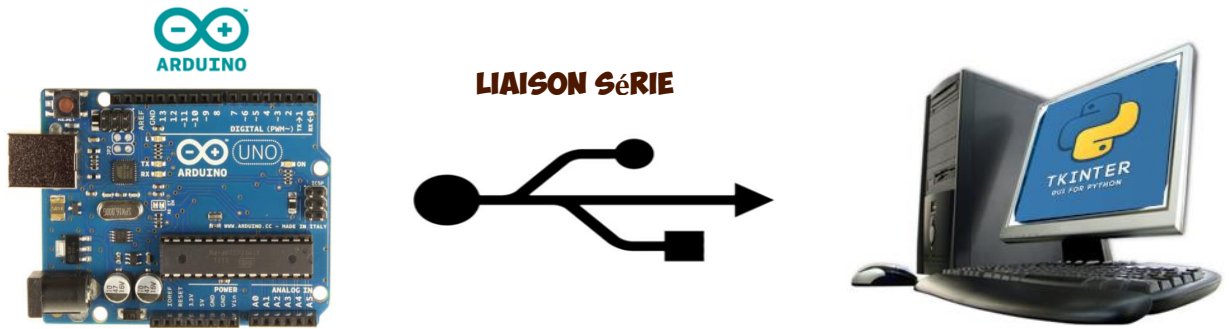
void loop()
{
  int luminosite = analogRead(A0) ;
  Serial.println(luminosite) ;
  delay(500) ;
}
```

- Enregistrer le programme sous le nom « **LSerie_CapteurLumière** », compiler et transférer le programme, ouvrir la console série pour voir les valeurs mesurées :
- Faire varier la luminosité reçue par la LDR en masquant la lumière reçue par le capteur avec la main, repérer les valeurs correspondant au « jour », et celles correspondant à la « nuit ». Déterminer le seuil de passage entre les 2 modes.
- Compléter le code afin de satisfaire au cahier des charges.
Enregistrer sous le nom « **LSerie_CapteurLumiere_CdC6** »



II. ARDUINO+PYTHON

1. CONFIGURATION MATERIELLE ET SYSTEME

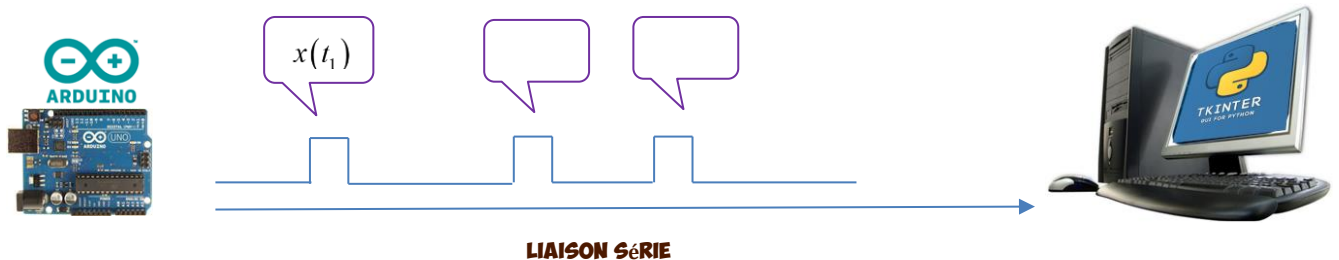


On utilise la carte Arduino et sa liaison série pour communiquer avec le PC mais on ne se contente plus ici de la console de la fenêtre arduino sur le PC.

Les données circulant sur la liaison sont maintenant **générées ou exploitées par** Python/TkInter : toute l'interactivité et toutes les possibilités graphiques de TkInter sont alors au service du pilotage et de l'exploitation de la carte Arduino.

2. RECEPTION ET TRAITEMENT D'UNE SEULE VARIABLE

Ici on cherche à envoyer des données variables mais de même nature : c'est donc une variable $x(t)$ qui dépend du temps t qui circule sur la liaison et qui doit être lue et interprétée par le programme Python du PC.



a. Côté Arduino

Reprendre le programme **LSerie_CapteurLumière**: il se contente donc de lire la valeur sur le capteur de lumière et de la renvoyer sur la liaison série.



Noter le numéro de port de communication utilisé par la carte Arduino.

b. Côté PC sous Python



Lancer l'interpréteur Python et ouvrir l'exemple **simpleLecture.py** :



Ce fichier contient le code minimal à l'utilisation de la liaison série par Python :



Il faut importer la librairie associée à la gestion d'une communication sur la liaison série :

```
import serial
```



```
serial_port = serial.Serial( port = "COM3" , baudrate = 9600)
```

Crée une instance de port série configurée correctement (bon numéro de port, débit identique à celui de la carte arduino)

Exo 3. Lancer **LSerie_CapteurLumière** sur la carte Arduino et sous Python, **simpleLecture.py** sur le PC

- Interpréter et expliquer les affichages console python en les confrontant aux instructions du programme Python.

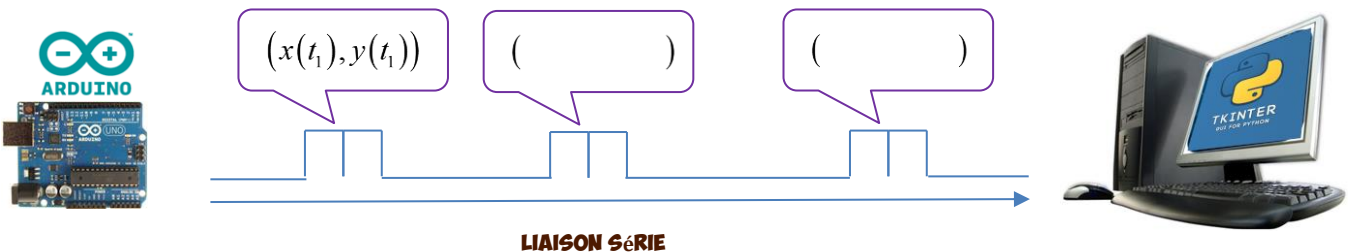
expliquer en particulier le rôle de l'instruction :

`chaineRecue=octetsRecus.decode()` #decode en chaine

- b. Si la carte n'émet aucune information sur la liaison série, quel est le comportement du programme ?
Apporter des modifications minimales au code qui permettent de mettre en évidence ce fonctionnement.

3. RECEPTION ET TRAITEMENT DE PLUSIEURS VARIABLES

Ici on cherche à envoyer des données variables mais de nature différente : c'est donc plusieurs variable $(x(t);y(t);...)$ qui dépendent du temps t qui circulent sur la liaison et qui doivent être lues et interprétées par le PC.



C'est donc une suite de couple de caractères qui circule sur la liaison.

a. Côté Arduino :

Exo 4. On envoie les valeurs en utilisant la virgule comme séparateur.

- Reprendre le programme **LSerie_CapteurLumière**: il se contente pour l'instant de lire la valeur sur le capteur de lumière et de la renvoyer sur la liaison série.
- Enregistrer ce programme sous le nom **LSerie_CapteurLumière_Rotation**
- Brancher le capteur d'angle de rotation (**Rotary Angle Sensor**) sur le connecteur analogique **A3**.
- Modifier le code du programme afin qu'il envoie les deux données séparées par une virgule avant un saut de ligne



La concaténation de chaînes n'est pas gérée par la fonction `Serial.println`.

L'instruction suivante (bien qu'elle soit tentante...) ne fonctionnera pas

```
Serial.println(luminosite+", "+angle) ;
```

Il faudra donc plusieurs instructions d'émission pour envoyer le couple des deux valeurs.

`Serial.println()` provoque un retour ligne, alors que `Serial.print()` se contente d'envoyer la chaîne sur la liaison série sans retour ligne.

b. Côté PC et interpréteur Python



Ouvrir l'exemple `doubleLecture.py` :

La structure du programme ne change pas. Seul le traitement déclenché par la réception de données est modifié de façon à interpréter correctement les différentes données reçues.

Exo 5.

- Expliquer a priori (en examinant le code) le fonctionnement du code.
Quel est le type de la variable **capteurs** ?
- Manipulation :
 - Côté Arduino : Compiler, télécharger.
 - Côté PC : Lancer `doubleLecture.py`
 - Les valeurs de luminosité sont lues et affichées par l'interpréteur Python.
Faire varier l'une et l'autre en manipulant les capteurs sur la carte Arduino
- Modifier le code pour déterminer les plages de valeurs décrites par l'angle de rotation et par la luminosité en exposant le capteur à la lumière ou dans l'obscurité.

III. EXPLOITATION GRAPHIQUE : ARDUINO + PYTHON + TKINTER

On dispose de trois images : Un décor en journée, un décor en nocturne, et un motard.

cdc 7. Le PC affiche dans la fenêtre graphique de TKInter le motard dans l'un des deux décors.

Le décor évolue en fonction de la luminosité ambiante.

En fonction de l'angle commandé à la main, le motard se déplace sur la largeur de la fenêtre (angle maximal : la moto disparaît sur la droite de l'image ; angle minimal : la moto disparaît à reculons sur la gauche de l'image).



Exemple d'affichage attendu sur le PC :



Le capteur de luminosité dit qu'il fait jour



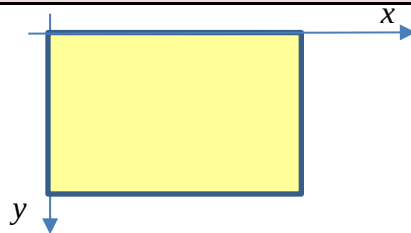
.... Le capteur de rotation est actionné....



... le capteur de luminosité signale la tombée de la nuit

- Choisir les 3 images que vous souhaitez utiliser.
- Coder les programmes nécessaires à la réalisation de ce cahier des charges sur la carte Arduino et sur Python/TKInter.

Rappel : Dans le canevas, les objets sont placés en utilisant un repère dans lequel l'origine est située à son coin supérieur gauche, les valeurs de x augmentant vers la droite et les valeurs de y augmentant vers le bas.



cdc 8. Le motard allume ses phares si vous actionnez le bouton poussoir.

- On peut créer une 4ème image à partir de celle du motard avec phares allumés (utiliser Gimp, PhotoShop,...)

cdc 9. Si le motard atteint les limites du décor, la diode rouge s'allume.

cdc 10. Une pression sur le bouton poussoir de la carte fait sauter le motard :

Une première version pourra **téléporter** le motard en haut de l'image et le maintenir en l'air une seconde.

Une deuxième version devra le **transporter** en haut de l'image dans une trajectoire presque « continue » (disons fluide) et qui suit les lois physiques de la gravité.

Si t est le temps, v_0 est la vitesse initiale impulsée par le bouton poussoir (!) et z est l'altitude,

$$z(t) = v_0 t - \frac{1}{2} g t^2. \quad (g \text{ est la constante de gravité : } g \approx 9,81 \text{ m/s}^2).$$

En pratique, on peut concevoir que l'altitude est donc une fonction du type $z(t) = \alpha t^2 + \beta t$ et ajuster les coefficients α et β en fonction du réalisme et du visuel attendus.

